

Lecture 29. Calculators

why scheme? Scheme have few rules, which means that we can use python to write all these thing

A programming language has:

- **Syntax:** The legal statements and expressions in the language
- **Semantics:** The execution/evaluation rule for those statements and expressions

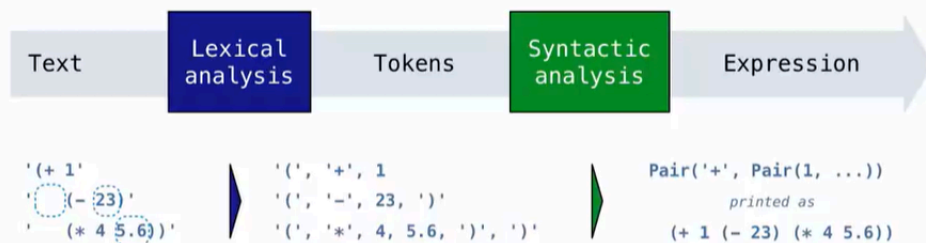
To create a new programming language, you either need a:

- **Specification:** A document describe the precise syntax and semantics of the language
- **Canonical Implementation:** An interpreter or compiler for the language

Parsing

Parsing

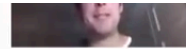
A Parser takes text and returns an expression.



- Iterative process
- Checks for malformed tokens
- Determines types of tokens
- Processes one line at a time

- Tree-recursive process
- Balances parentheses
- Returns tree structure
- Processes multiple lines

Recursive Syntactic Analysis



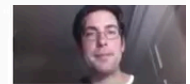
A predictive recursive descent parser inspects only k tokens to decide how to proceed, for some fixed k .

Can English be parsed via predictive recursive descent?

sentence subject
 The horse ~~raced~~ past the barn fell.
 ↑ ridden
 (that was)

```
> ((1 2) 3)
((1 2) 3)
Pair(Pair(1, Pair(2, nil)), Pair(3, nil))
> █
```

```
95 def scheme_read(src):
96     """Read the next expression from src, a Buffer of tokens.
97
98     >>> lines = ['(+ 1 ', '(+ 23 4)) (']
99     >>> src = Buffer(tokenize_lines(lines))
100     >>> print(scheme_read(src))
101     (+ 1 (+ 23 4))
102     """
103     if src.current() is None:
104         raise EOFError
105     val = src.pop()
106     if val == 'nil':
107         return nil
108     elif val not in DELIMITERS: # ( ) ' .
109         return val
110     elif val == "(":
111         return read_tail(src)
112     else:
113         raise SyntaxError("unexpected token: {}".format(val))
114
115 def read_tail(src):
116     """Return the remainder of a list in src, starting before an element or ).
117
118     >>> read_tail(Buffer(tokenize_lines([' ']))
119     nil
120     >>> read_tail(Buffer(tokenize_lines(['2 3'])))
121     Pair(2, Pair(3, nil))
122     >>> read_tail(Buffer(tokenize_lines(['2 (3 4)'])))
123     Pair(2, Pair(Pair(3, Pair(4, nil)), nil))
124     """
125     if src.current() is None:
126         raise SyntaxError("unexpected end of file")
127     if src.current() == ")":
128         src.pop()
129         return nil
130     first = scheme_read(src)
131     rest = read_tail(src)
132     return Pair(first, rest)
133
```



Calculator

The Pair Class

The Pair class represents Scheme pairs and lists. A list is a pair whose second element is either a list or nil.

```
class Pair:
    """A Pair has two instance attributes:
    first and second.

    For a Pair to be a well-formed list,
    second is either a well-formed list or nil.
    Some methods only apply to well-formed lists.
    """
    def __init__(self, first, second):
        self.first = first
        self.second = second

>>> s = Pair(1, Pair(2, Pair(3, nil)))
>>> print(s)
(1 2 3)
>>> len(s)
3
>>> print(Pair(1, 2))
(1 . 2)
>>> print(Pair(1, Pair(2, 3)))
(1 2 . 3)
>>> len(Pair(1, Pair(2, 3)))
Traceback (most recent call last):
...
TypeError: length attempted on improper list
```

Scheme expressions are represented as Scheme lists! *Homoiconic* means source code is data.

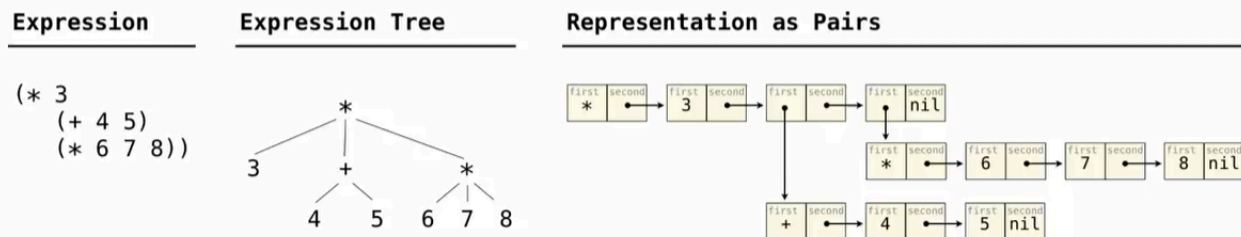
Calculator Syntax

The Calculator language has primitive expressions and call expressions. (That's it!)

A primitive expression is a number: 2, -4, 5.6

A call expression is a combination that begins with an operator (+, -, *, /) followed by 0 or more expressions: (+ 1 2 3), (/ 3 (+ 4 5))

Expressions are represented as Scheme lists (Pair instances) that encode tree structures.



Calculator Semantics

The value of a calculator expression is defined recursively.

Primitive: A number evaluates to itself.

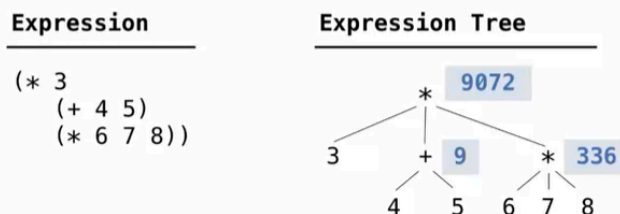
Call: A call expression evaluates to its argument values combined by an operator.

+: Sum of the arguments

***: Product of the arguments**

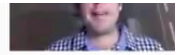
-: If one argument, negate it. If more than one, subtract the rest from the first.

/: If one argument, invert it. If more than one, divide the rest from the first.



Evaluation

The Eval Function



The eval function computes the value of an expression, which is always a number.

It is a generic function that dispatches on the type of the expression (primitive or call).

Implementation

```
def calc_eval(exp):  
    if type(exp) in (int, float):  
        return exp  
    elif isinstance(exp, Pair):  
        arguments = exp.second.map(calc_eval)  
        return calc_apply(exp.first, arguments)  
    else:  
        raise TypeError
```

Recursive call returns a number for each operand

'+', '-', '*', '/'

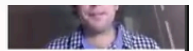
A Scheme list of numbers

Language Semantics

A number evaluates...
to itself

A call expression evaluates...
to its argument values
combined by an operator

Applying Built-in Operators



The apply function applies some operation to a (Scheme) list of argument values.

In calculator, all operations are named by built-in operators: +, -, *, /

Implementation

```
def calc_apply(operator, args):  
    if operator == '+':  
        return reduce(add, args, 0)  
    elif operator == '-':  
        ...  
    elif operator == '*':  
        ...  
    elif operator == '/':  
        ...  
    else:  
        raise TypeError
```

(Demo)

Language Semantics

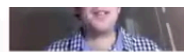
+:
Sum of the arguments

-:
...

...
...

Interactive Interpreters

Read-Eval-Print Loop



The user interface for many programming languages is an interactive interpreter.

- Print a prompt.
- **Read** text input from the user.
- Parse the text input into an expression.
- **Evaluate** the expression.
- If any errors occur, report those errors, otherwise
- **Print** the value of the expression and repeat.

Raising Exceptions

Raising Exceptions

Exceptions are raised within lexical analysis, syntactic analysis, eval, and apply.

Example exceptions

- **Lexical analysis:** The token 2.3.4 raises `ValueError("invalid numeral")`
- **Syntactic analysis:** An extra) raises `SyntaxError("unexpected token")`
- **Eval:** An empty combination raises `TypeError("( ) is not a number or call expression")`
- **Apply:** No arguments to - raises `TypeError("- requires at least 1 argument")`

Handling Exceptions

An interactive interpreter prints information about each error.

A well-designed interactive interpreter should not halt completely on an error, so that the user has an opportunity to try again in the current environment.